

Online Event Booking & Smart Ticketing Platform

High-Level Architecture Overview | React.js + Tailwind | Node.js + Express | PostgreSQL

1. Purpose & Scope

This document describes the high-level architecture of a web platform where **Attendees** discover and book events, **Event Organizers** publish and run events, and **Admins** govern the marketplace. It covers the main system components, how the frontend, backend and database interact, the authentication and role flow, and the five core modules. Schema design, API contracts and code are intentionally out of scope.

Architectural goals

- **No double-booking:** seat inventory must stay consistent under heavy concurrent demand (e.g. a popular event going on sale).
- **Trustworthy money flow:** payments are confirmed server-side by the gateway, and every rupee is traceable through invoices, commission and payouts.
- **Tamper-proof tickets:** QR codes cannot be forged or reused at the gate.
- **Clear role separation:** each role sees only its own data and actions.
- **Simple to build, ready to scale:** start as a modular monolith with clean module boundaries that can be split into services later if needed.

Technology stack at a glance

Layer	Technology	Responsibility
Frontend	React.js, Tailwind CSS, React Router, Axios, a server-state library (e.g. TanStack Query)	Role-based UI, seat map, checkout, ticket wallet, dashboards, QR scanner
Backend	Node.js, Express.js, JWT, validation (Joi/Zod), ORM/query builder (Prisma/Knex/Sequelize)	Business rules, auth & RBAC, booking engine, payment orchestration, reporting APIs
Database	PostgreSQL	System of record with ACID transactions for bookings and payments
Supporting	Redis, job queue (BullMQ), S3-compatible storage, payment gateway, email/SMS provider	Seat holds, caching, background jobs, file storage, payments, notifications

2. System Architecture

The platform follows a classic three-tier layout. A React single-page app talks to a stateless Express API over HTTPS; the API owns all business logic and is the only component that reads or writes PostgreSQL. Redis, a job queue and external providers support the core without holding the source of truth.

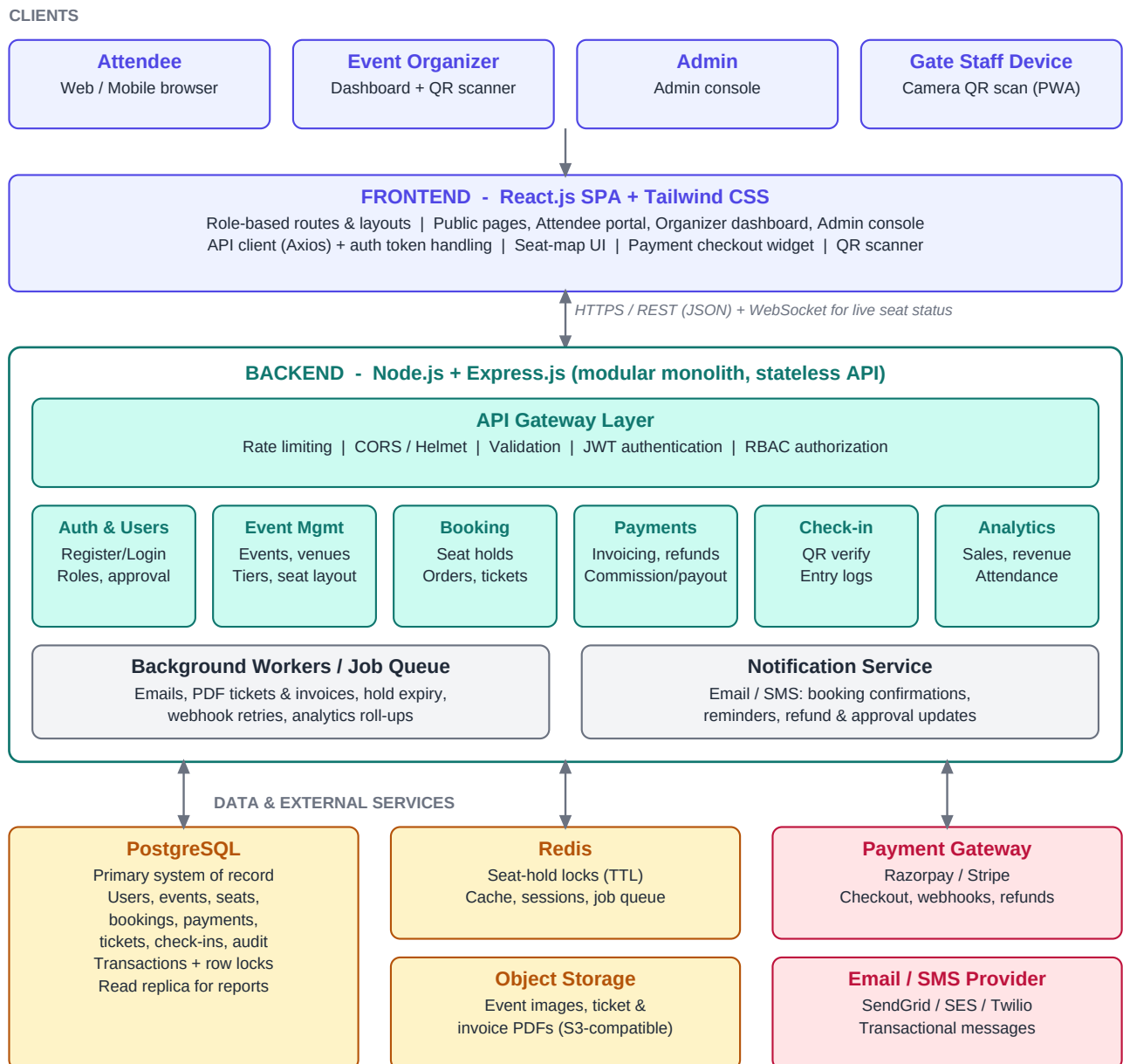


Figure 1 - High-level component view

2.1 How the tiers interact

- **Frontend -> Backend**: the SPA calls versioned REST endpoints (`/api/v1/...`) with a JWT access token. A WebSocket (or short polling) channel pushes live seat availability while a user is on the seat map.
- **Backend -> Database**: Express modules access PostgreSQL through a data-access layer. Booking and payment writes run inside database transactions so that seats, orders and tickets always change together.
- **Backend -> Redis**: short-lived seat holds, caching of hot read data (event listings, categories) and the job queue.
- **Backend <-> Payment gateway**: the backend creates a payment order; the browser completes checkout; the gateway calls the backend *webhook*, which is the only trusted signal that payment succeeded.

- **Workers:** slow or retry-able tasks (PDF generation, emails, hold expiry, analytics roll-ups) run outside the request cycle so user-facing APIs stay fast.

2.2 Frontend architecture

- **Four app areas in one SPA:** Public (home, search, event details), Attendee portal (bookings, tickets, invoices, refunds), Organizer dashboard (events, attendees, check-in, sales) and Admin console (approvals, oversight, settings, disputes, analytics).
- **Route guards:** protected routes check login state and role before rendering; the backend still enforces every rule.
- **Shared UI kit:** Tailwind-based components (cards, tables, forms, charts, seat grid) keep all portals visually consistent.
- **Server state:** API data is cached and refreshed through a query library; lightweight global state holds the session and cart.
- **Gate scanner:** a mobile-friendly page using the device camera to read QR codes, usable as a PWA by organizer staff.

2.3 Backend architecture

- **Modular monolith:** one deployable Express app organised by domain module (auth, events, booking, payments, check-in, analytics, admin). Each module has its own routes, controllers, services and data access.
- **Layered request pipeline:** security middleware -> validation -> authentication -> authorization -> controller -> service (business rules) -> repository (database).
- **Stateless API:** no session state in memory, so multiple instances can run behind a load balancer.
- **Domain events:** modules announce key events (e.g. *BookingConfirmed*, *RefundApproved*) that trigger notifications, invoices and analytics updates without tight coupling.
- **Audit log:** sensitive actions (approvals, refunds, commission changes, check-in overrides) are recorded with who, what and when.

2.4 Data layer

- **PostgreSQL** is the single source of truth, grouped into domains: identity & roles, organizer profiles, venues & seat layouts, events & ticket tiers, orders & tickets, payments/refunds/payouts, check-ins, and audit records.
- **Integrity through the database:** foreign keys, unique constraints (a seat can be sold only once per event) and transactions with row-level locking protect correctness even if application code has a bug.
- **Reporting:** heavier analytics read from summary tables or materialized views (optionally on a read replica) so dashboards never slow down booking traffic.
- **Files** (event banners, ticket and invoice PDFs) live in object storage; the database stores only their references.

3. Authentication & Role Flow

All three roles share one identity system. Each user account has exactly one role, and the role decides which routes, screens and data the user can reach.

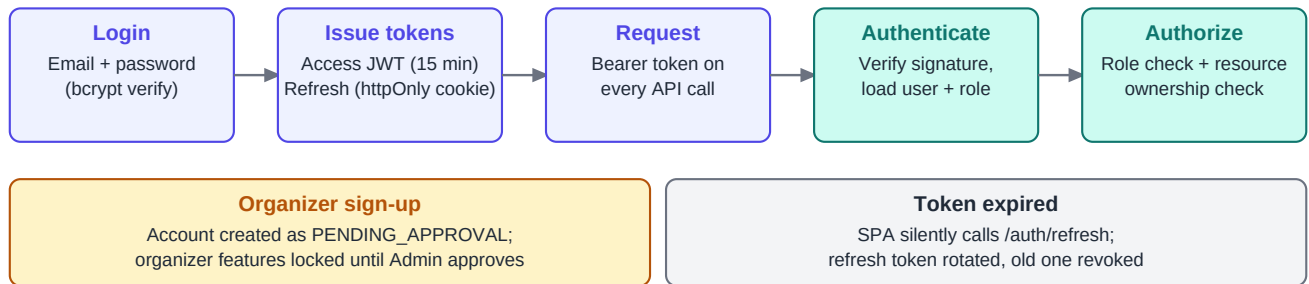


Figure 2 - Authentication and authorization flow

- **Registration:** Attendees become active after email verification. Organizers register with business details and stay *Pending Approval* until an Admin approves them; Admins are created only by existing Admins (never via public sign-up).
- **Tokens:** a short-lived access JWT (carrying user ID and role) plus a long-lived refresh token stored in an httpOnly, secure cookie. Refresh tokens are rotated and can be revoked on logout, password change or account suspension.
- **Two-level authorization:** (1) *role check* - is this role allowed to call this endpoint? (2) *ownership check* - does this resource belong to the user? e.g. an Organizer can only see attendee lists for their own events.
- **Account states:** Active, Pending Approval, Suspended. Suspended users are blocked at the authentication step.

Role-to-capability matrix

Module	Attendee	Event Organizer	Admin
Auth & Profile	Register, login, manage profile	Register (needs approval), business profile, payout details	Approve / reject / suspend organizers, manage users
Event Management	Browse, search, filter, view details	Create, edit, publish, cancel own events; tiers & seat layout	View/moderate all events, feature or unpublish
Booking	Select seats/tiers, book, view history	View bookings and attendee list for own events	View all bookings platform-wide
Payment & Invoicing	Pay, download invoice, request refund	View revenue and payout statements for own events	Set commission & payout rules, resolve disputes, approve refunds
QR Check-in	Hold digital ticket with QR code	Scan and verify tickets at entry, view check-in stats	View check-in logs, override in disputes
Analytics	-	Event-wise sales, tiers sold, attendance rate	Platform revenue, commission earned, popular events, attendance trends

4. Key Modules

4.1 Event Management

Owns everything an Organizer publishes and everything an Attendee searches.

- **Event lifecycle:** Draft -> Published -> Sales Closed -> Completed, with Cancelled possible at any point (which triggers bulk refunds).
- **Venues & seat layouts:** organizers define a layout once (sections, rows, seats, or general-admission zones) and reuse it across events. Each event gets its own copy of seat inventory.
- **Ticket tiers & pricing:** e.g. VIP, Gold, General, Early Bird - each with price, quantity, sale window and per-order limit.
- **Discovery:** search and filters by category, date range, city/location and price, powered by PostgreSQL indexes and full-text search, with popular listings cached in Redis.
- **Admin oversight:** admins can view, feature, unpublish or flag any event on the platform.

4.2 Seat / Ticket Booking

The most concurrency-sensitive part of the system. Its core job is to guarantee that a seat is never sold twice.

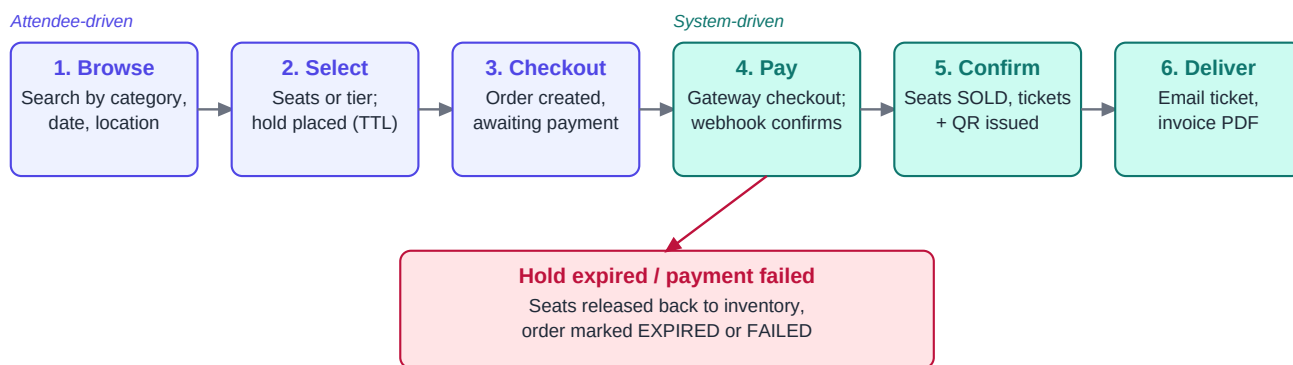


Figure 3 - Booking lifecycle

- **Temporary seat hold:** when an Attendee selects seats, the system reserves them for a short window (e.g. 10 minutes) using an atomic lock with a time-to-live. Other users see those seats as unavailable.
- **Final guarantee in the database:** on payment confirmation, a single transaction marks seats sold, creates tickets and confirms the order. A unique constraint makes a duplicate sale impossible even in rare race conditions.
- **Tier-based (non-seated) events** decrement an available-quantity counter inside the same transaction instead of locking seats.
- **Expiry:** a background job releases holds and marks unpaid orders as expired, returning seats to inventory.
- **Cancellation:** Attendees can cancel within the event's refund policy; this releases seats and opens a refund request.
- **Order states:** Pending Payment, Confirmed, Expired, Failed, Cancelled, Refunded (partial or full).

4.3 Payment & Invoicing

- **Gateway integration:** Razorpay (suited to India: UPI, cards, net banking) or Stripe. Card data never touches our servers.
- **Server-side confirmation:** the booking is confirmed only after the gateway's signed webhook is verified. Webhook handling is idempotent, so duplicate deliveries never create duplicate tickets.

- **Invoicing:** after confirmation, a worker generates a GST-ready invoice PDF with a sequential invoice number, stores it and links it to the booking history for download.
- **Commission & payouts:** each sale is split into platform commission (rule set by Admin - global or per organizer) and organizer earnings. Earnings are settled to organizers after the event on a payout schedule, with a statement per event.
- **Refunds & disputes:** Attendee raises a request -> auto-approved if within policy, otherwise routed to the Admin dispute queue -> refund issued through the gateway -> commission and payout adjusted -> Attendee notified. Event cancellations refund all buyers in bulk.
- **Ledger mindset:** payments, refunds, commissions and payouts are stored as immutable records so totals can always be reconciled.

4.4 QR Check-in / Verification

- **Ticket issuance:** each confirmed ticket gets a unique ID and a QR code containing a *digitally signed token* (ticket ID + event ID, signed with a server secret). It is shown in the Attendee's ticket wallet and attached to the email.
- **Verification at the gate:** organizer staff scan with the scanner page -> backend verifies the signature, checks the ticket belongs to that event and to the scanning organizer, and confirms it is valid and not yet used.
- **Single use:** check-in is recorded atomically, so a second scan of the same code shows *Already checked in* with the original time and gate - screenshots or copies cannot be reused.
- **Clear results:** Valid (green), Already Used (amber), Invalid / Wrong Event / Cancelled (red), each with attendee name and tier.
- **Resilience:** the scanner can pre-download the event's ticket list for areas with poor connectivity and sync check-ins later.
- **Live stats:** organizers see checked-in vs expected attendees in real time.

4.5 Analytics

- **Organizer analytics (per event):** tickets sold by tier, gross sales, net earnings after commission, sales over time, refunds, and attendance rate from check-in data.
- **Admin analytics (platform-wide):** total revenue and commission, top events and categories, organizer performance, booking and attendance trends, refund rates.
- **How it is computed:** domain events and scheduled jobs update summary tables / materialized views (e.g. daily sales per event), so dashboards read pre-aggregated data instead of scanning raw bookings.
- **Presentation:** charts and KPI cards in React, with date-range filters and CSV/PDF export.

5. Cross-Cutting Concerns

Concern	Approach
Security	HTTPS everywhere, bcrypt password hashing, JWT with refresh rotation, Helmet and CORS rules, input validation, parameterized queries (SQL injection safe), rate limiting on login and booking APIs, signed QR tokens, webhook signature checks, secrets in environment variables.
Data protection	Role + ownership checks on every request, audit log for sensitive actions, minimal personal data exposed to organizers, regular encrypted backups.
Performance	Redis caching for listings, database indexes on search fields, pagination everywhere, background jobs for slow tasks, CDN for the frontend and images.
Scalability	Stateless API instances behind a load balancer, read replica for analytics, queue-based workers scaled independently, and module boundaries that allow later extraction into services.
Reliability	Transactions for money and inventory, idempotent webhooks and retries, scheduled reconciliation with the payment gateway, health checks.
Observability	Structured logging, request IDs, error tracking (e.g. Sentry), uptime and performance monitoring.
Notifications	Event-driven emails/SMS: registration, organizer approval, booking confirmation with ticket, reminders, cancellation and refund updates.

6. Deployment View

- **Frontend:** React build served as static files from a CDN or hosting service (e.g. Vercel, Netlify, S3 + CloudFront).
- **Backend:** containerised Express API (Docker) running on a cloud VM or container service, with 2+ instances behind a load balancer.
- **Workers:** separate process/container reading from the Redis-backed job queue.
- **Data:** managed PostgreSQL (automated backups, optional read replica) and managed Redis.
- **Environments:** Development -> Staging (gateway sandbox mode) -> Production, with CI/CD running tests and migrations on each release.

7. Suggested Delivery Phases

Phase	Focus
1. Foundation	Auth, roles, organizer approval, event creation and public browse/search
2. Booking & Payments	Ticket tiers, seat layout and holds, checkout, webhook confirmation, digital tickets
3. Post-booking	Invoices, booking history, cancellation and refund workflow, notifications
4. Event Day	QR scanner, verification, check-in logs and live attendance
5. Insights & Governance	Organizer and admin analytics, commission and payout management, disputes, audit